FLIGHTSIM · PHASE 3

Phase 3 Guidelines

Consolidation, component-level annotation, and a demonstration anyone can watch. The step-by-step plan and the guideline sheet, revision 2.

> For Shashank Aitharaju · Dhruva Valluru
> Revision 2 · 27 October 2026 · branch `phase-2-final`
> Built from: the Phase 3 requirements (13 Oct), the Windows review (24–25 Oct), and the A-4 model decode

WINDOWS the check needs a Windows machine with the engine installed

PYTHON Python-only work that may start while Phase A is open

OWNER waits on a decision by Shashank or Dhruva

CHECKPOINT stop and re-verify before going on

Contents

1. **Part 1 · The guideline sheet**: tick-boxes in plain words, letters A–F, with the gates
2. **Part 2 · The full implementation plan**: every task with its files, what to do and how to verify it

PART 1

The guideline sheet

Do the letters in order. Tick a box only when its check passes.

The plain-words version of `tasks/phase3_todo.md` (revision 2, 2026-10-27). The letters match the plan: **A**1 here is A1 there.

How to use this sheet

- Do the letters in order: A → B → C → D → E → F.
- Do not start a letter until the gate before it is ticked.
  - **One exception:** the Python-only boxes in C (marked `PYTHON`) may be done while A is still open.
- Tick a box only when its **check** passes. Change `[ ]` to `[x]` and commit.
- `WINDOWS` means the check needs a Windows machine with the engine installed.
- If a check fails, write what you saw in `docs/PHASE3_LOG.md` before fixing it.
- The rule for the whole phase: **do less, and make what remains work for someone who is not us.** If a box turns out expensive, say so; it may be cut.

Why the order changed (the weekend Windows review)

What works: everything in Python. Plan in plain English, reproducible campaigns, the verifier, the exporter refusing bad data, and the aircraft sitting 0.4 m from its label (it was ~25 m).

What doesn't: the Unreal half. Each problem below has its own box in A.

What happened on Windows	Box that fixes it
Build failed on 5.5 (the code names a 5.7-only setting)	A1, A2, A3
Had to import the aircraft by hand	A7
Had to skip host flight with `--no-host-flight`	A8
An old September program drew the pictures	A4
Only one of two cameras gave pictures	A9
Generate in the web app always refused	A10
Tight box empty, "alone" pictures missing (3 of 7 label checks fail)	A11, A12
The whole aircraft is one colour in the mask (no gear or parts)	C (payload and gear); control surfaces are not this phase

A · Make the Unreal half work, with no workarounds

- [ ] **A1 · Pick one engine version and install it on all three computers** — *you decide*
  - Recommended: go back to **5.5** (it is what actually runs). Or keep 5.7, but then all three computers install the same 5.7 first.
  - Check: `docs/TOOLCHAIN.md` has three rows with the same version, and a dated decision line.
- [ ] **A2 · That version written in every place the project names it**
  - Check: `pytest -q tests/test_platform.py` passes.
  - Check: changing the version in one file by hand makes that test fail.
- [ ] **A3 · The build refuses the wrong engine** WINDOWS
  - Check: pointing it at another version stops with `engine.version_mismatch`.
  - Check: the right version builds with zero errors.
- [ ] **A4 · An old engine program can't draw the pictures**
  - Check: after changing one `.cpp` without rebuilding, a render stops with `engine.binary_stale`.
- [ ] **A5 · JSBSim plugin set up again on this version** WINDOWS
  - Check: `scripts/check_bridge_api.sh` passes and the build passes.
  - Check: each of its six patches is marked needed or not needed.
  - Check: the plane flies the same in the engine as in Python (`gate5_ue_parity.py`).
- [ ] **A6 · Materials build by themselves** WINDOWS
  - Check: the first run prints `MATERIAL-CREATED` for the five rebuilt ones, and the second run prints `MATERIAL-EXISTS`.
- [ ] **A7 · The aircraft is imported automatically** WINDOWS
  - Check: delete the imported A-4, press Generate, and it imports and renders by itself.
- [ ] **A8 · Host flight works; nobody needs `--no-host-flight`** WINDOWS
  - Check: a web-app run with host flight completes, and no message tells the user to add the flag.
- [ ] **A9 · Every camera gives pictures, or the run says which one didn't** WINDOWS
  - Check: the default run has the same number of frames for both cameras.
  - Check: an empty camera stops with `render.camera_empty`.
- [ ] **A10 · Generate works in the web app** WINDOWS
  - Check: a one-line prompt on `/generate.html` finishes a campaign.
- [ ] **A11 · The engine writes the label pictures; missing ones fail loudly** WINDOWS
  - Check: every `render.json` has the mask, depth and an "alone" picture per aircraft.
  - Check: a run with missing label pictures stops with `labels.no_bundle`.
- [ ] **A12 · All seven label checks pass; one real frame saved as a test** WINDOWS
  - Check: `python -m flightsim.verify runs/<id>` shows 7 of 7 PASS.
  - Check: `label_diagnose.py` shows `tight null: 0`.
  - Check: `pytest -q tests/test_engine_bundle_golden.py` passes.
- [ ] **A13 · NEXT.md says only what was measured**

GATE A

- [ ] Tests pass on all three operating systems.
- [ ] **On the reviewer's computer, from a fresh download:** Generate gives pictures from both cameras and 7 of 7 label checks pass, with no hand import, no flag and no old program.

- [ ] The same on the other two computers.

B · Lock the tools

- [ ] **B1 · Exact Python package versions saved with hashes** (`requirements.lock`)
  - Check: a fresh install from it, then `pytest -q` , passes.
- [ ] **B2 · One file lists every other tool's version** (`toolchain.json`)
  - Check: preflight prints `OK` per tool, and a wrong version stops it.
- [ ] **B3 · A test catches any version drifting**
  - Check: `pytest -q tests/test_toolchain_pins.py` passes.
- [ ] **B4 · Picture quality checked on this engine** WINDOWS
  - Check: `gate6_visual.py` passes, and the A12 label test still passes.

GATE B

- [ ] A fresh download builds with nothing set except `toolchain.json` , and preflight is all `OK` on all three computers.

C · Boxes around the payload (and optionally the gear)

Good news: the A-4's model already contains its bombs and tanks as separate pieces. They are just switched off today, so this does not wait for a new aircraft.

Two rules to keep in mind:

- **The aircraft's own box still includes its payload.** The payload gets its own colour in the mask, but it is still part of the plane.
- **Python decides every part's ID; the engine only follows.**

- [ ] **C1 · The A-4 file lists its payload and a default loadout** PYTHON — *you decide the loadout*
  - Check: the A-4 loads with at least one payload, and the plane still trims with the weight of what it carries.
- [ ] **C2 · Each loaded station exported as its own piece**
  - Check: the mesh manifest lists a `payload` piece per loaded station.
- [ ] **C3 · The run card lists the payload pieces with their own IDs**
  - Check: the IDs are unique and never clash with other aircraft.
- [ ] **C4 · The engine gives each piece its own colour in the mask** WINDOWS
  - Check: the mask contains the payload's ID, and the aircraft's own tight box is still filled.
- [ ] **C5 · Payload boxes written into the label file (manifest v8)** PYTHON
  - Check: `pytest -q tests/test_component_labels.py` passes for five cases: visible, off-screen, hidden behind the plane, aircraft box includes payload, and old v7 files still read.
- [ ] **C6 · The checker and exports know about payload** PYTHON
  - Check: `component_coverage` passes, and a COCO export has a `payload` category.
- [ ] **C7 · (Optional) Landing gear boxes, on the c172p** (its gear is fixed and visible; the A-4's gear is stowed)
  - Check: folded-up gear is "not claimed", never a zero-size box.

GATE C

- [ ] A web-app A-4 run passes verify with `component_coverage` PASS. WINDOWS
- [ ] Nobody pressed an extra button or added a flag.

D · New aircraft and places

- [ ] **D1 · Survey: which Chinese aircraft we are allowed to use** — *you decide*
  - So far there is no J-15, J-35 or J-11 in FlightGear's official add-on collection. One early Su-27SK project includes a J-11A; its licence still needs checking.
  - Recommended: J-11A only, or **cut it** if the licence isn't clean.
- [ ] **D2 · First new aircraft added** (with payload from day one)
  - Check: its config test passes, it flies level in a headless run, and it imports by itself.
- [ ] **D3 · (If two were chosen) second new aircraft**
- [ ] **D4 · Desert and Forest buttons in the app**
  - Check: Desert gives Grand Canyon and Forest gives Yosemite.
- [ ] **D5 · The two places are prepared once and reused** — *you decide whether typing the Google tiles key once is acceptable*
  - Check: a second launch says "up to date" for both.
- [ ] **D6 · Every aircraft × place combination runs with no special tweaks** WINDOWS
  - Check: every combination passes verify and has payload boxes.

GATE D

- [ ] The new aircraft is available, both places work, payload boxes are present, and the old aircraft still pass. WINDOWS

E · Make it simple for someone new

- [ ] **E1 · Every setting listed and sorted** into: in the app / hidden default / developers only / delete
  - Check: a test finds no unlisted setting.
- [ ] **E2 · All settings in one file**, edited from the app
  - Check: an empty settings file gives working defaults, and Google tiles are still on.
- [ ] **E3 · Settings panel in the app**
  - Check: the tiles key survives a page reload, and 6 or fewer fields are visible.
- [ ] **E4 · Pressing Capture with nothing typed gives a working run** — *you decide whether storm weather is in the demo default*
  - Check: the run finishes and verify has no FAIL.
- [ ] **E5 · Hidden options become checkboxes** (normals / flow / amodal)
- [ ] **E6 · Double-click launcher** (`Launch FlightSim.cmd`) WINDOWS
  - Check: on a fresh download, the app opens and the log ends with `READY`.
  - Check: a second launch skips everything already done.

GATE E

- [ ] Someone new uses only the launcher and the app and gets a passing run. They don't read NEXT.md, open a terminal or ask you anything. WINDOWS
- [ ] Their questions are written in `docs/PHASE3_LOG.md`.

F · Demo and wrap-up

- [ ] **F1 · Automated full run:** pick aircraft → pick place → Capture → download WINDOWS
  - Check: `pytest -q -m ue_host tests/test_demo_e2e.py` passes.
- [ ] **F2 · The same run works on all three computers** WINDOWS
  - Check: three passing rows in `docs/PHASE3_LOG.md` with the same tool versions.
- [ ] **F3 · Short guide written** (`docs/GUIDE.md`)
  - Check: under 1,000 words, with no terminal commands.
- [ ] **F4 · Video of one full run**, under 5 minutes
- [ ] **F5 · Final report:** each of the six goals with its proof

The six goals (the brief's "done" list)

- [ ] 1. A new person does a full capture without help or the notes.
- [ ] 2. A J-15, J-11 or J-35 is available, and the same pipeline works in Desert and Forest.
- [ ] 3. Payload boxes appear automatically wherever the payload is visible.
- [ ] 4. One full run (configure, capture, label) with no manual steps, on locked tool versions.
- [ ] 5. A short video and a short guide exist.
- [ ] 6. Nothing a user does needs the command line.

Not this phase

- Separate mask colours for ailerons, elevator, rudder and flaps (control surfaces).
- World-wide terrain.
- Supporting two engine versions at once.

Decisions waiting on you or Shashank

- [ ] **Engine:** back to 5.5 (recommended), or 5.7 on all three computers. *This one blocks everything.*
- [ ] Which payload the A-4 carries by default
- [ ] Which Chinese aircraft, or cut it
- [ ] Storm weather on in the demo, or off?
- [ ] Is typing the Google tiles key once acceptable?
- [ ] Keep control surfaces out of Phase 3?

PART 2

The full implementation plan

Each task: the files it touches, what to do, and how to verify it.

Source: *Phase 3 — Requirements* (13 October 2026). Theme: **pull back — refine and consolidate rather than extend.** Revision 2 written 2026-10-27 on `phase-2-final` at b95e711e. It folds in two things the first revision (2026-10-18) did not have:

1. **The Windows review of the weekend of 24–25 October.** The Python half works as delivered. The Unreal half does not, on any machine but one.
2. **A decode of the A-4 model** showing its stores already exist as separate parts.

This file is a plan only. No code has been written for it yet. The plain-words companion is `tasks/phase3_checklist.md`.

0 · What the Windows review found (the evidence this plan is built on)

Works as delivered (Python):

- A sentence goes in and a plain-English plan comes back.
- A campaign runs reproducibly across worker counts.
- The verifier grades labels against the pixels independently and names what fails.
- The exporter refuses a dataset whose checks did not pass.
- The Phase 1 placement defect is fixed: the aircraft sits 0.4 m from its labelled position along the flight axis, measured from the mask, against ~25 m before. The refusal text quotes the old failure.

Does not work (Unreal):

#	Symptom	Cause, checked in this tree	Fixed by
R1	The build fails on a 5.5 machine	`ue/Source/FlightSim.Target.cs:9` and `FlightSimEditor.Target.cs:9` set `EngineIncludeOrderVersion.Unreal5_7`, which 5.5 does not have. Nothing on the reviewer's machines has compiled against 5.7	A1–A3
R2	The aircraft mesh had to be imported by hand	No automatic import: `flightsim/capture.py:337` and `webapp/generate.py:137` refuse and tell the user to run `scripts/import_aircraft.py`	A7
R3	Host flight had to be skipped with `--no-host-flight`	Cause in the review document. `flightsim/capture.py:1263` offers the flag in its refusal text	A8
R4	The September Phase 1 binary drew the frames	Nothing checks that the engine binary was built from the current source	A4
R5	Only one of the two cameras produced frames	Cause in the review document	A9
R6	Generate in the web app refuses every time	Cause in the review document	A10

#	Symptom	Cause, checked in this tree	Fixed by
R7	`bbox_2d_tight` is null in every record	The `-labels` bundle is never complete, so `attach_engine_labels` (`core/capture/labels.py:1670`) fills nothing; this is reported as a progress line, not a failure	A11
R8	Per-object "alone" passes are never written	The same uncompiled commandlet (`FlightSimRenderCommandlet.cpp` ~L4300–4365)	A11
R9	One mask ID for the whole aircraft: gear, ailerons and control surfaces are not segmented	The aircraft is drawn as one stencil value (`FlightSimRenderCommandlet.cpp:2876-2877`); the A-4 is exported as one rigid body	C1–C7 (payload, gear); control surfaces deferred, see §Not in scope

R1, R7 and R8 together are why three of the seven annotation gates fail.

Note: the review document itself did not reach the session that wrote this plan. R3, R5 and R6 cite it for their cause. A8–A10 start by reproducing each one from that document.

From the A-4 decode (2026-10-27):

- `A4E.mdl` gates its stores behind `STAn_LOAD` / `STAn_RACK` / `STAn_RAIL` visibility conditions.
- Loading stations 1–3 adds one or two separate parts and about 1,800–14,600 triangles per station.
- `assets_pipeline/a4_mdl_convert.py` exports with every store off (`mdl_scene.REST_ENV`), as one body.
- So payload work does not wait for a new airframe.

Repository context

Layer	What it is	Where
Physics	JSBSim 1.2.4 (Python wheel), 120 Hz	`core/fdm` , `core/control`
Capture and labels	Python. The manifest is v7. Per-frame `labels.objects[]`	`core/capture/labels.py` , `manifest.py` , `schema.py` , `verify.py`
Engine half	Unreal C++ plugin plus a render commandlet. Custom-stencil ID pass (8-bit, aircraft is `int_id` 1). Object IDs come from the card (`FlightSimScenarioWorld.cpp:1233`)	`ue/Plugins/FlightSimBridge/…`
User interface	FastAPI web app plus static pages (`index.html` , `generate.html` , `frames.html`)	`webapp/`
Aircraft	One JSON per subject. FlightGear `.ac` meshes, except the A-4 (decoded P3D `.mdl`)	`assets/aircraft_config/` , `assets_pipeline/`
Installer	`scripts/deploy_windows.ps1` : git, Python, clone, venv, server, browser. No engine build, materials, aircraft import or terrain	`scripts/`
Tests	pytest, `ue_host` marker for engine-only tests, mutation guards (`scripts/mutation_check.sh` , never at the same time as pytest)	`tests/`

Size key: XS = 1 file · **S** = 1–2 files · **M** = 3–5 files. "Windows" means the step needs a render machine with the engine installed. Every other check runs in a Linux container or in CI.

Owner's standing rules this plan keeps (CLAUDE.md):

- Google tiles are the ground of every terrain render.
- Web-app storm weather is on by default.
- No scheduled PR check-ins.

Where a task runs into one of them, it is flagged `OWNER` rather than changed silently.

Order and why:

1. **A** · The engine half works with no workarounds.
2. **B** · Lock the tools.
3. **C** · Component boxes.
4. **D** · New subjects and places.
5. **E** · Consolidate the user path.
6. **F** · Demo and close-out.

Consolidation (E) comes after the features (C, D) so the interface is designed once, around what actually exists. **The Python half of C (C1, C5, C6 on synthetic bundles) may run alongside A.** Every engine step waits for Gate A.

Phase A — The Unreal half builds and renders, with no workarounds

The brief's first sequencing item. Each R-symptom above becomes a named fix, because each one is a "manual intervention" the brief forbids.

A1 · Choose one engine and install it on all three machines — XS · `OWNER`

- **Files:** `docs/TOOLCHAIN.md` (new)
- **Decide:** the 2026-10-19 decision was 5.7, for render quality. The review shows 5.7 has not built on the reviewer's machines, and the brief says "if a newer engine cannot be relied on, pin back to what can".
  - **Recommended:** pin back to **5.5** for Phase 3. It is installed, and every measurement before October was taken on it.
  - **Alternative:** keep 5.7, but only if all three machines install the same 5.7 patch before any other A-step.
  - **Not offered:** building on both versions at once, which doubles the test surface.
- **Do:** record per machine the exact engine build (`Engine/Build/Build.version` Major.Minor.Patch and Changelist), the MSVC toolset and the Windows SDK. All three rows show the same version.
- **Below,** `<PIN>` means the version chosen here.
- **Verify:** three rows in `docs/TOOLCHAIN.md` with an identical engine version, and a dated decision line naming who decided.

A2 · One pin, everywhere, tested — M

- **Files:** `ue/FlightSim.uproject`, `ue/Source/FlightSim.Target.cs`, `ue/Source/FlightSimEditor.Target.cs`, `core/util/platform.py`, `tests/test_platform.py`
- **Do:** set `EngineAssociation`, both `IncludeOrderVersion` constants and `UE_ENGINE_VERSION` to `<PIN>`. Add `UE_ENGINE_PATCH` for the A1 patch level.
- **If `<PIN>` is 5.5:** revert or version-guard the 5.7-only changes made in October:
  - the C4883 silencing on the commandlet's `Main`;
  - `GetMaterialResource(EShaderPlatform)`;
  - any other 5.7-only API call. Find them with `git log -S"5.7" -- ue/`.

- **Add a source test:** the `IncludeOrderVersion` constant names `<PIN>`. R1 becomes a CI failure, not a Windows surprise.
- **Verify:** `pytest -q tests/test_platform.py` passes. Hand-editing one Target.cs to another version makes it fail with the file named.

A3 · The build refuses the wrong engine by name — S

- **Files:** `scripts/build_ue.ps1`, `scripts/ue_preflight.ps1`
- **Do:**
  - Remove the "newest `UE_5.*` found" fallback (`build_ue.ps1:13-17`).
  - Resolve only `UE_ROOT` or the pinned install path.
  - Read `Engine/Build/Build.version` and refuse with `engine.version_mismatch: found X, pinned Y` before calling `Build.bat`.
- **Verify:**
  - Windows: `UE_ROOT` at another version → non-zero exit and `engine.version_mismatch`.
  - Pinned engine → exit 0, and the version it built against is printed.
  - Container: a source-reading test asserts the glob is gone.

A4 · A stale engine binary is refused (R4) — M

- **Files:** `scripts/build_ue.ps1`, `core/util/platform.py`, `flightsim/capture.py`, `webapp/runs.py`, `core/messages/catalog.yaml`
- **Do:**
  - After a successful build, write `ue/Plugins/FlightSimBridge/Binaries/build_stamp.json` with the engine version and a hash of the plugin's `Source/` tree.
  - Before any render, compare it with the current source. A missing or different stamp refuses with `engine.binary_stale`, naming both hashes.
  - The launcher (E6) uses the same stamp to decide whether to rebuild.
- **Verify:**
  - Container: `pytest -q -k binary_stale`. A synthetic stamp with a wrong hash refuses; a matching one passes.
  - Windows: touching one `.cpp` without rebuilding makes the next render refuse.

CHECKPOINT `pytest -q` is green. Windows: preflight finds the pinned engine, the build is clean (zero `error C`), and the stamp is written.

A5 · The JSBSim plugin on the pinned engine — S

- **Files:** `scripts/vendor_ue_plugin.ps1`, `ue/Plugins/JSBSimFlightDynamicsModel/VENDORED.json`
- **Do:** vendor against `<PIN>`. For each of the six `local_patches` (four from `scripts/jsbsim_plugin_patches_5_6.diff` plus two bridge additions), record whether it is still needed, still applied, or no longer applies. Write `ue_engine_target: "<PIN>"`.
- **Verify:**
  - Windows: `bash scripts/check_bridge_api.sh` exits 0 and the build is clean.
  - `experiments/gate5_ue_parity.py`: a c172p hold agrees with the Python flight within the recorded bound.

A6 · Label and look materials compile on the pinned engine — S

- **Files:** `scripts/ue_create_materials.py`

- **Do:** already partly done in b95e711e. Assets without the current `GRAPH_GENERATIONS` stamp rebuild themselves. Confirm the script runs unattended and is idempotent on `<PIN>`, and that `M_CustomStencilID` needs no editor step.
- **Verify:** Windows. The first run prints `MATERIAL-STALE` → `MATERIAL-CREATED` for M_LandcoverID, M_LensDrops, M_GreyCard, M_RainStreaks and M_Ground_City. The second run prints `MATERIAL-EXISTS` for all of them, and the editor log has no "Missing If" errors.

A7 · The aircraft mesh is imported automatically (R2) — M

- **Files:** `flightsim/capture.py` (~L337, ~L797), `webapp/generate.py` (~L137), `webapp/runs.py`, `scripts/import_aircraft.py`
- **Do:** where the code now refuses with "run `scripts/import_aircraft.py`", run the import itself, once, and log it. Import when:
  - the mesh is missing;
  - or its `mesh_manifest.json` is older than the aircraft config or converter.

  The refusal stays only for a real failure, such as a missing licence file or converter error. - **Verify:** - Windows: delete the imported A-4 asset, press Generate, and the run imports it and renders. - Container: a test with a missing mesh and a fake importer shows the import is called, not the refusal.

A8 · Host flight works, and `--no-host-flight` leaves the user path (R3) — M

- **Files:** `flightsim/capture.py` (~L1221, L1263), `core/capture/hostflight.py`, the host C++ named by the review
- **Do:**
  1. Reproduce the host-flight failure from the review document and fix its cause.
  2. Remove the flag from the refusal text, which must name the fault instead.
  3. Keep the flag as developer-only (E1).
- **Known related open item (NEXT.md):** `FlightSimScenarioWorld.cpp` ~2650 `Commands.Elevator += Input.Elevator` accumulates across entries. Fix it here if the review's failure is this, or record it.
- **Verify:** Windows. A web-app run with host flight on completes, and the parity check against the Python flight passes. `grep -n "no-host-flight" webapp flightsim/capture.py` shows no user-facing text.

A9 · Every camera renders, or the run fails by name (R5) — S

- **Files:** `FlightSimRenderCommandlet.cpp`, `flightsim/capture.py`, `webapp/runs.py`
- **Do:**
  1. Reproduce the empty second camera from the review document and fix it.
  2. After the render, a camera with zero frames refuses with `render.camera_empty: <camera id>`.
- **Verify:** Windows. The default two-camera run writes the same frame count for both cameras. Container: a synthetic run with one empty camera directory refuses by name.

A10 · Generate works in the web app (R6) — S

- **Files:** `webapp/generate.py`, `webapp/server.py`, the cause the review names
- **Do:** reproduce the refusal from the review document and fix its cause. The likely chain is the A7/A8 refusals surfacing through `words()`, so confirm before changing anything.
- **Verify:**
  - Windows: `/generate.html` takes a one-line prompt to a finished campaign.
  - Container: `tests/test_generate*.py` gains a case reproducing the review's refusal, which now passes.

A11 · The label bundle is written; no bundle is a failure (R7, R8) — M

- **Files:** `FlightSimRenderCommandlet.cpp` (stencil ~L2876, mask writer ~L4385, alone passes ~L4300–4365), `core/capture/labels.py`, `webapp/runs.py`, `flightsim/capture.py`, `core/messages/catalog.yaml`, `tests/test_camera_labels.py`
- **Do:**
  - Every `-labels` frame writes `frames/<camera>/render.json` with a `labels` block: `mask`, `depth_f32`, and `objects[].alone_png` for each aircraft.
  - After `attach_engine_labels`, a run that requested a render and got `attached == 0` or `without_bundle > 0` refuses with `labels.no_bundle`, naming the camera and the frame count.
  - A headless run keeps today's behaviour.
- **Verify:**
  - Container: `pytest -q tests/test_camera_labels.py -k no_bundle`.
  - Windows: one render. Every `render.json` has `mask`, `depth_f32` and `objects`, the mask is not all zeros, and an alone PNG exists per aircraft per frame.

A12 · All seven annotation gates pass; one real frame becomes a fixture — S

- **Files:** `core/capture/verify.py`, `scripts/label_diagnose.py`, `tests/fixtures/engine_bundle_min/` (new), `tests/test_engine_bundle_golden.py` (new)
- **Do:**
  - Add a verifier check: when the aircraft is in frame and a bundle exists, `bbox_2d_tight` is non-null and lies inside `bbox_2d`.
  - `label_diagnose.py` prints null and filled tight-box counts per camera.
  - Commit the smallest passing run (1 camera, 1 frame, low resolution) and pin its tight box, visible fraction and depth median.
- **Verify:**
  - Windows: `python -m flightsim.verify runs/<id>` gives all seven gates PASS, and `label_diagnose.py` prints `tight null: 0`.
  - Container: `pytest -q tests/test_engine_bundle_golden.py`, with the fixture under 2 MB.

A13 · NEXT.md says only what is measured — XS

- **Files:** `NEXT.md`
- **Do:**
  - Rewrite gotcha 32 and the October "FIRST WINDOWS RUNS" text with the review's findings and A1–A12's measured results.
  - Mark the 5.7 claims as measured on one machine only, unless A1 chose 5.7 and all three now pass.
  - Cite the A12 run id and the fixture path.
- **Verify:** no statement in NEXT.md contradicts `docs/TOOLCHAIN.md` or the review.

CHECKPOINT **Gate A:**

- CI is green on all three operating systems.
- **On the reviewer's machine, from a fresh clone:**
  - Generate in the web app gives frames from both cameras;
  - all seven annotation gates PASS;
  - no hand import, no `--no-host-flight`, no old binary.
- Repeated on the other two machines.

No engine-side Phase 3 work starts before Gate A.

Phase B — Lock the tools (Req. 4)

B1 · Python dependencies locked with hashes — S

- **Files:** `requirements.lock` (new, `pip-compile --generate-hashes`), `.github/workflows/ci.yml`, `scripts/deploy_windows.ps1`
- **Do:** CI and the installer install from the lock. `requirements.txt` stays the human-edited source.
- **Verify:** in a fresh venv, `pip install --require-hashes -r requirements.lock && pytest -q` passes on all three CI operating systems.

B2 · One manifest for every other tool — M

- **Files:** `toolchain.json` (new: engine `<PIN>` and patch, MSVC, Windows SDK, Cesium for Unreal, ffmpeg, Python), `scripts/ue_preflight.ps1`, `core/util/platform.py`
- **Do:** `platform.py` reads the pin from `toolchain.json`, replacing the A2 literal. Preflight compares each installed tool and refuses by name with `toolchain.<tool>_mismatch`.
- **Verify:** Windows. Preflight prints one `OK <tool> <version>` per entry, and a wrong ffmpeg version in the file makes it exit non-zero.

B3 · A test keeps the pins in one place — XS

- **Files:** `tests/test_toolchain_pins.py`
- **Do:** assert that `toolchain.json` == `EngineAssociation` == both `IncludeOrderVersion` == `UE_ENGINE_VERSION`, and that `requirements.lock` covers `requirements.txt`.
- **Verify:** it passes; changing one pin by hand fails with the file named.

B4 · Picture quality re-measured on the pinned engine — M

- **Files:** `experiments/gate6_visual.py`, `ue/Config/DefaultEngine.ini`, `core/sky/plan.py`, `docs/VALIDITY.md`
- **Do:**
  - If `<PIN>` is 5.5, confirm the existing 5.5 numbers still hold.
  - If it is 5.7, re-derive the palette and exposure biases from probe renders.
  - Labels do not depend on the look, so this cannot break A11–A12.
- **Verify:** Windows. `gate6_visual.py` (measure and beauty) passes, `gate10_render_repro.py` gives a bounded verdict, and the A12 fixture still passes.

CHECKPOINT **Gate B:** a fresh clone builds with nothing set beyond `toolchain.json`, and preflight is all `OK` on all three machines.

Phase C — Component-level annotation (Req. 3; payload required, gear optional)

Conventions are reused, not duplicated. A component is one more entry in the frame's `labels.objects[]`, with:

- a `parent` (the aircraft's `id`) and a `class_id` of `payload` or `landing_gear`;

- the same keys as the aircraft: `bbox_2d_hull`, `fraction_in_frame`, `bbox_2d_tight`, `visible_fraction`, `occluded_by`, `not_claimed`, `basis`;
- the same null-with-basis rule when out of frame or hidden.

Two rules that do not exist yet and that C must add:

1. **Parent rollup.** The ID pass holds one value per pixel, so a payload pixel is no longer an aircraft pixel. The aircraft's `bbox_2d_tight`, `visible_fraction` and mask therefore count its own components' pixels. A component never appears in its own parent's `occluded_by`. Without this rule the aircraft box silently shrinks the moment payload is added.
2. **The card declares components; the engine follows.** Python allocates component `int_id`s in a band that cannot collide with traffic (for example 64–127) and writes them on the card. The C++ reads them as it reads every other object (`FlightSimScenarioWorld.cpp:1233`) and invents none.

C1, C5 and C6 run on synthetic bundles and may start during Phase A.

C1 · `components` block and a default loadout — S · [OWNER: the loadout]

- **Files:** `core/capture/airframe.py`, `assets/aircraft_config/A4.json`, `assets/fdm_root/aircraft/A4/A4.xml`
- **Do:**
  - Add an optional `components: [{id, class: payload|landing_gear, station?, source}]` block, and an A-4 `loadout` (for example a centreline drop tank on STA3 and one store each on STA2/STA4).
  - **Physics and picture agree:** the loaded stations' FDM point-mass weights are set to the drawn stores (they are 0 lb today, `loadable: false`), and the run trims with them.
  - The loader refuses a component that names no part (`aircraft.component_unmatched`).
- **Verify:**
  - `pytest -q tests/test_airframe.py -k components` loads the A-4 with at least one payload, and a dead station is refused by name.
  - A headless A-4 run with the loadout trims and verifies with no FAIL.

C2 · The converter exports each loaded station as its own mesh — M

- **Files:** `assets_pipeline/a4_mdl_convert.py`, `assets_pipeline/mdl_scene.py`, `tests/test_a4_mesh.py`
- **Do:**
  - Evaluate the scene with `REST_ENV` plus each loaded station's `STAn_LOAD` / `RACK` / `RAIL`. The body keeps the rest-environment parts; each station's added parts become their own OBJ.
  - `mesh_manifest.json` gains `components[]` (id, class, extents, origin) and a version bump; a stale manifest is re-converted.
- **Verify:** `python -m assets_pipeline.a4_mdl_convert assets/aircraft_config/A4.json`, then the manifest has a `payload` component per loaded station whose triangle count matches the measured delta (STA3 load 1: +14,612 over rest).

C3 · The card carries component objects — S

- **Files:** `core/scenario/card.py`, `core/capture/labels.py` (the annotation block)
- **Do:** each component is written to the card's object list with `int_id` (from the band), `parent`, `class` and its mesh name.
- **Verify:** `pytest -q -k card_components`. The A-4 card lists the components with unique IDs in the band, and a traffic aircraft's ID never collides with them.

C4 · The engine stencils components and draws the parent's alone pass whole — M · Windows

- **Files:** `scripts/ue_import_aircraft.py`, `FlightSimRenderCommandlet.cpp`, `FlightSimScenarioWorld.cpp`, `FlightSimScenarioWorld.h`
- **Do:**
  - Import one `SM_A4_<component>` per component and attach it under the aircraft's `MeshOrigin`.
  - Set each component's stencil from the card.
  - The aircraft's alone pass draws the aircraft and its components together, so the parent's amodal box includes its stores.
  - Each component gets its own alone pass.
- **Verify:** Windows. One A-4 still with `-labels`:
  - `render.json` `labels.objects[]` has at least one object with `parent` set and `class == "payload"`;
  - `numpy.unique(mask)` contains its `int_id`;
  - the aircraft's tight box is still filled (A12 unchanged).

CHECKPOINT `pytest -q` is green, including the A12 fixture unchanged.

C5 · Component label records, parent rollup, manifest v8 — M

- **Files:** `core/capture/labels.py` (`object_label_record` ~L807, `measure_object` ~L1140, `attach_engine_labels` ~L1670), `core/capture/manifest.py`, `core/capture/schema.py`, `docs/schemas/capture_manifest.v8.schema.json` (new), `tests/test_component_labels.py` (new)
- **Do:**
  - Emit component entries. `bbox_2d_hull` and `fraction_in_frame` come from the component extents projected through the same camera; the engine keys come from its `int_id`.
  - `measure_object` for a parent treats the IDs {parent} ∪ children as its pixels, and drops children from `occluded_by`.
  - Set `MANIFEST_VERSION = 8`, keeping 3–7 readable. `conventions()` documents `payload`, `landing_gear` and the rollup.
- **Verify:** `pytest -q tests/test_component_labels.py` covers these cases:
  1. a payload blob gives a tight box equal to the blob;
  2. an out-of-frame payload gives `bbox_2d_tight: null` with the out-of-frame basis;
  3. a payload behind the fuselage gives `visible_fraction == 0` and `occluded_by == [<aircraft id>]`;
  4. the aircraft's tight box includes its payload pixels;
  5. `validate_manifest` accepts v8 and v7.

C6 · Verifier and exporters know the components — M

- **Files:** `core/capture/verify.py`, `core/dataset/export.py`, `tests/test_dataset_boxes.py`, `tests/test_annotation_gates.py`
- **Do:**
  - The verifier checks that each component's tight box lies within its own hull and its parent's tight box.
  - **`component_coverage`:** every frame with payload pixels in the mask carries a payload record. This is acceptance criterion 3.
  - COCO, YOLO and VOC export `payload` (and `landing_gear`) as their own categories; the independent readers re-read them.
- **Verify:** `pytest -q -k component` passes, and a mutant that drops the payload record fails `component_coverage`.

C7 · Landing gear (optional; only if C1–C6 are green) — S

- **Files:** `assets/aircraft_config/c172p.json` (fixed gear, named `.ac` parts), `assets_pipeline/convert.py`
- **Do:** split the gear parts named by the existing `gear_geometry` into a `landing_gear` component. **Not on the A-4:** its gear is stowed and not articulated (`a4_mdl_convert.py` header). Retracted gear on any type is `not_claimed` with basis `gear retracted`, never a zero-size box.
- **Verify:** Windows. A c172p render gives a `landing_gear` tight box on every frame whose mask contains its ID.

CHECKPOINT **Gate C:**

- Windows: a web-app A-4 run passes `python -m flightsim.verify` with `component_coverage` PASS.
- `dataset.zip` opens in COCO with a `payload` category.
- No user step was added.

Phase D — New subjects and environments (Req. 2)

D1 · Source survey and go/no-go — XS · OWNER

- **Files:** `docs/PHASE3_AIRCRAFT_SOURCES.md` (new)
- **Do:** one row per candidate, giving the mesh source, licence and pinned commit, the JSBSim FDM source, whether payload and gear are separable parts, and the keypoint provenance.
- **What a first search found (2026-10-27):** no J-15, J-35 or standalone J-11 in FGAddon. An early-development Su-27SK project (github.com/yanes19/Su-27SK, v0.3, 2017) lists a J-11A variant; its licence is unchecked.
- **Recommendation:** J-11A only, if its licence is clean. Otherwise **cut D2–D3** and say so now, per the brief.
- **Verify:** every column is filled or marked `none found`, with a dated owner decision line.

D2 · First new aircraft — M

- **Files:** `assets/aircraft_config/J11.json`, `assets/fdm_root/aircraft/J11/…`, `tests/test_aircraft_configs.py`
- **Do:** the config carries dimensions, the seven `KEYPOINT_NAMES`, `components` (payload from day one, per Phase C), `limits`, `licence` and `gear_geometry`. The FDM must trim at the default run's altitude and speed.
- **Verify:** `pytest -q tests/test_aircraft_configs.py -k J11`. A headless default run trims and verifies. Windows: the A7 auto-import brings it in with no command.

D3 · Second new aircraft (only if D1 chose two) — S

- **Do:** same as D2. A shared Su-27-family FDM or engine file is reused, not copied.

D4 · Desert and Forest presets — S

- **Files:** `webapp/runs.py` (`SCENE_SETTING_BAKES` ~L1260), `webapp/static/index.html`
- **Do:** a two-option environment picker over the existing rule (desert → `grand_canyon`, forest → `yosemite`). A preset fixes location, surface, sun and season. Google tiles stay the ground (CLAUDE.md). Global terrain is out of scope.
- **Verify:** `pytest -q -k environment`. "Desert" gives `grand_canyon` / `desert` on the card, and "Forest" gives `yosemite` / `forest`.

D5 · The two places are prepared once — S

- **Files:** `scripts/bake_terrain.py`, the launcher (E6)
- **Do:** pre-bake both GLO-30 tiles on first launch, check the sidecar sha256, and never re-stream after that.
- OWNER Google tiles still stream at render time with a key. Is entering it once acceptable as "no manual intervention"?
- **Verify:** a second launch logs `bake grand_canyon up to date` and `bake yosemite up to date`.

D6 · The same pipeline, unchanged, across the matrix — M

- **Files:** `scripts/run_test_scenarios.py`, `tests/test_phase3_matrix.py` (new), `docs/PHASE3_LOG.md`
- **Do:** run {A-4, new aircraft} × {Desert, Forest} from the default run, changing only the aircraft and environment choice.
- **Verify:** every cell is `verify PASS` with `component_coverage` PASS, and the cells' cards differ only in `aircraft` and `scene`.

CHECKPOINT **Gate D:** acceptance criteria 2 and 3 are met on Windows, and the A12 fixture is still green.

Phase E — Consolidate the user path (Req. 1, 6)

E1 · Inventory the configuration surface — S

- **Files:** `core/registry.py`, `docs/CONFIG_SURFACE.md` (new)
- **Do:** list every `FLIGHTSIM_*` variable (20 read today in core/webapp/flightsim: LLM, LLM_MODEL, SKY, WEATHER_BACKEND, RENDER_QUALITY, GOOGLE_TILES, RENDER_TIMEOUT_SECONDS, FFMPEG, STALL_SECONDS, RAIN_STYLE, FONT, CAPTURE_PASSES, XPLANE_TERRAIN/SKY/LIGHTING, TERRAIN_PREFETCH, RELAY_URL, RAIN_GAIN, GEOCODER, GEOCODE_CACHE), plus `UE_ROOT` and every CLI flag the user path reaches, including `--no-host-flight`. Each gets a decision:
  - **UI:** stays exposed, in the interface;
  - **default:** hidden, value fixed;
  - **dev-only:** tests and debugging scripts only;
  - **remove.**
- **Verify:** a registry test finds every `FLIGHTSIM_…` name registered with a decision.

E2 · One settings file — M

- **Files:** `core/registry.py`, `webapp/server.py`, `webapp/runs.py`, `tests/test_settings.py` (new)
- **Do:** `settings.json` in the user data directory is the only source for the UI items, behind `GET/PUT /settings`. Environment variables become developer overrides, recorded in the run as `env override`. The owner's defaults stay: tiles on, storm weather on.
- **Verify:** an empty file and an empty environment give the documented defaults, and `FLIGHTSIM_GOOGLE_TILES=off` is still honoured.

E3 · Settings panel — S

- **Files:** `webapp/static/index.html`, `webapp/static/settings.js` (new)
- **Do:**
  - Show at most six fields: tiles key, Cesium status, engine (read-only, from preflight), render quality, LLM key, and the extra passes.

    - Everything else goes under "Advanced" or is removed.
    - A missing required item shows as a named, fixable message.
- **Verify:** a headless-Chromium page test sets the key, reloads, and finds it persisted.

E4 · Capture with nothing typed gives a working run — S · [OWNER: storm weather in the default]

- **Files:** `webapp/runs.py`, `webapp/server.py`
- **Do:** `/run {}` defaults to the A-4 with its loadout, Desert, the chase camera, a short clip and labels on. The Capture button submits it.
- **Verify:** headless `POST /run {}` reaches `done` with no FAIL. Windows: the same request renders and verifies.

E5 · Extra passes become checkboxes — S

- **Files:** `webapp/capture.py`, `webapp/static/index.html`
- **Do:** `FLIGHTSIM_CAPTURE_PASSES` (normals, flow, amodal) becomes checkboxes, and the dev-only variables are no longer read on the web path.
- **Verify:** `grep -n "FLIGHTSIM_CAPTURE_PASSES\|FLIGHTSIM_XPLANE" webapp/*.py` shows only the registry read.

E6 · Double-click launcher — M

- **Files:** `Launch FlightSim.cmd` (new), `scripts/deploy_windows.ps1` (extended, not replaced), `scripts/ue_preflight.ps1`
- **Do:** in order, stopping at the first failure with a named reason in the window:
    1. Install or update from `requirements.lock` (what `deploy_windows.ps1` already does).
    2. Preflight against `toolchain.json`.
    3. Build, only if the A4 stamp does not match.
    4. Create materials, only if one is missing or stale.
    5. Import aircraft, only if stale (A7).
    6. Bake the two places (D5).
    7. Start the server and open the browser.
- **Verify:**
    - Windows, fresh clone: double-click, and `logs/launch.log` ends with `READY`.
    - A second launch logs `up to date` for the build, materials, aircraft and bakes.
    - Container: a source test asserts the step order and that no step prompts.

CHECKPOINT **Gate E (acceptance 1 and 6):** someone other than the two authors clones the repo, double-clicks the launcher, presses Capture and gets a verified run. They never touch NEXT.md, a terminal or an environment variable. Their questions are recorded in `docs/PHASE3_LOG.md`.

Phase F — Demonstration and close-out (Req. 4, 5)

F1 · Automated end-to-end run — S

- **Files:** `tests/test_demo_e2e.py` (new, `ue_host`)
- **Do:** drive the real page through these steps, with no other input:

1. Pick the aircraft.
2. Pick Desert.
3. Press Capture.
4. Wait for `done`.
5. Download `dataset.zip`.

CI runs the same test headless, up to the render step. - **Verify:** Windows. `pytest -q -m ue_host tests/test_demo_e2e.py` passes, and the zip has payload boxes wherever there are payload pixels.

F2 · Three-machine reproduction — XS

- **Files:** `docs/PHASE3_LOG.md`
- **Do:** run F1 from a fresh clone on all three A1 machines, recording the machine, preflight output, run id, verify result, wall time and the hash of `toolchain.json` + `requirements.lock`.
- **Verify:** three rows, all `verify PASS`, with the same hash.

F3 · The short guide — S

- **Files:** `docs/GUIDE.md` (new), `README.md`
- **Do:** what to open, what to choose, what to expect, and what the output contains (including the aircraft and payload boxes). The README's first screen links it, and NEXT.md is off the user path.
- **Verify:**
  - `wc -w docs/GUIDE.md` < 1000.
  - `grep -nE '\bpython\b|\.ps1|\$env:|FLIGHTSIM_|--[a-z]' docs/GUIDE.md` is empty.

F4 · Recorded run — XS

- **Files:** `docs/demo/README.md` (link and checksum; the video lives outside git if over 20 MB)
- **Do:** screen-record one F1 run, from the double-click to opening the dataset.
- **Verify:** duration ≤ 300 s, and the run id is in `PHASE3_LOG.md` with `verify PASS`.

F5 · Acceptance sheet — XS

- **Files:** `docs/PHASE3_REPORT.md` (new)
- **Do:** one row per acceptance criterion with its evidence (task, run id, test), plus what was cut and why.
- **Verify:** six rows, and no evidence cell is empty.

Acceptance criteria → tasks

#	Criterion (brief)	Closed by
1	A newcomer completes a capture without the notes	A7–A10, E1–E6, Gate E, F3
2	J-35/J-15/J-11 available; same pipeline in desert and forest	D1–D6
3	Payload boxes in the manifest wherever visible	A11–A12 (the path), C1–C6
4	Hands-off end-to-end run on a pinned toolchain	A1–A6, B1–B4, F1–F2
5	Short video plus a guide short enough to read	F3, F4
6	No command line on the user path	A7, A8, E3, E5, E6, F3

Not in scope this phase

- **Control-surface segmentation** (ailerons, elevator, rudder, flaps; review R9). The component mechanism in C supports it later, but the brief asks for payload and optional gear only. The A-4 would also need an articulated export first.
- **Global terrain, worldwide imagery and large-area streaming** (brief §2).
- **Building on two engine versions at once.**
- **The two overlapping weather paths** (NEXT.md): left as they are unless they break the demo run.

Owner decisions this plan waits on

1. **A1 · the engine:** pin back to 5.5 (recommended), or keep 5.7 with the same patch on all three machines. **This is the critical path.**
2. **C1 · the A-4 default loadout.**
3. **D1 · which Chinese airframe:** J-11A only if its licence is clean, or cut.
4. **E4 · storm weather in the demo default:** CLAUDE.md makes it the web-app default, but it is the least-proven render path.
5. **D5 · the Google tiles key** entered once in Settings: acceptable as "no manual intervention"?
6. **Scope:** keep control surfaces out of Phase 3?